

TD VAT flottant : Complexité algorithmique

La complexité représente la substantifique moëlle d'un problème. Chaque problème possède sa propre complexité intrinsèque.

1 Complexité d'un algorithme vs. d'un problème

Soit T un tableau d'entiers de taille n , **non trié**. Voici un algorithme testant si le tableau possède une valeur E donnée.

```
Inclue (T : tableau de n entiers, E : entier)
-- Renvoie vrai ssi  $E < \max(T)$ 
Pour i de 1 à n
    Si  $T(i) = E$  retourner vrai
Retourner faux
```

Ici, ce qui peut varier est n , la taille du tableau : 10 ou 1 milliard ? Les valeurs numériques sont assumées codées sur des entiers logant dans des registres, 32 ou 64 bits par exemple. Par conséquent, nous supposons que les opérations de comparaison (instruction dans la boucle pour, ou même traitement de l'indice de boucle) ont des durées constantes. La taille de l'entrée est $(2+n) \times \text{taille entier}$, taille du tableau, entier E , et tableau lui-même. Tout étant constant à part n , nous disons donc que l'entrée a une taille n donnée par la taille du tableau.

- a. Quelle est la complexité de cet algorithme en fonction de n ?

Maintenant observons le problème, peut-on faire mieux ? Par exemple, la mémoire cache effectue cette recherche de clé à chaque requête à la RAM. Pensez-vous qu'elle parcourt les valeurs de clés stockées en cache une par une ? Non, une mémoire associative permet, de façon matérielle, une recherche en parallèle sur toutes les clés. Une mémoire associative n'est pas une architecture Von Neumann. Il est complexe de modéliser un ordinateur complet, Alan Turing a proposé une abstraction très simple, que l'on nomme **Machine de Turing** :

Une machine de Turing est définie par une tête de lecture/écriture sur un ruban infini à gauche et à droite de la tête. Le ruban est constitué de cases pouvant contenir des symboles de l'alphabet ou le symbole blanc (noté b ou ϵ), qui remplit toutes les cases non initialisées du ruban. Ce ruban correspond à une mémoire séquentielle. La tête suit un algorithme qui est en fait une machine à états, qui dépend du symbole lu, et de l'état courant. En fonction de cela, la transition donne le symbole à écrire, et l'action à effectuer : un pas à gauche L , ou un pas à droite R . Un état spécial, **Halt**, est atteint lorsque c'est un état final de la machine à états, ou bien si aucune transition de l'état courant avec le symbole lu n'existe (cela correspondrait à un plantage). On peut par exemple imaginer un comptage en base 1, comme sur cet exemple, tiré d'un TD du Dr. Steven J. Zeil.

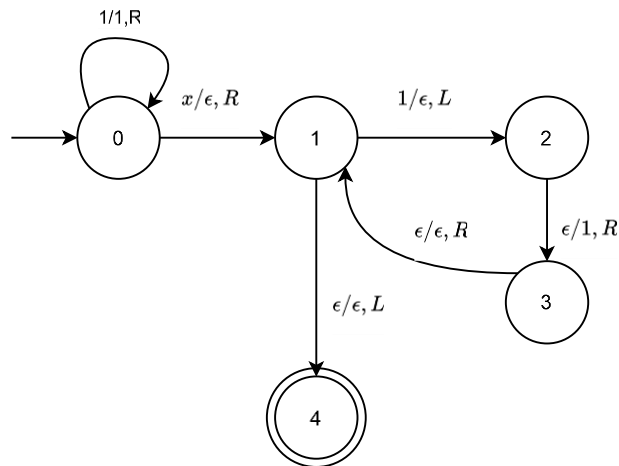


Figure 1 : Système d'instructions d'une machine de Turing additionnant deux nombres en base 1. L'entrée est donnée sur le ruban à droite de la tête, sous forme 111x11 par exemple pour 3+2, ici nous devons obtenir 11111.

- b. Vérifier que pour l'entrée 111x11, avec la tête sur le premier 1, on obtient bien 11111 avec la tête sur le dernier 1.
- c. Qu'ajouter pour que la tête soit positionnée sur le premier 1 ?
- d. Que se passe-t-il si on a un symbole non reconnu en entrée ?
- e. Supposons que nous voulions faire un programme Incrémenter, mais d'un entier donné en décimale. La tête est initialement placée sur le chiffre de poids faible, à droite du ruban. Le programme doit incrémenter l'entier et placer à la fin la tête sur le chiffre de poids faible. Donner le programme sous forme d'un automate comme sur la Figure 1.

La Machine de Turing ne va pas nous servir à calculer une complexité, car l'absence d'un accès direct à la mémoire (une bande est une mémoire séquentielle) va augmenter le nombre d'opérations nécessaires. Cependant, elle est équivalente en puissance d'expression à une Random Access Machine qui représente registres, RAM et opérations de base d'un microprocesseur. C'est sur ce type de modèle que la complexité d'un programme est assumée.

La Machine de Turing a notamment été utilisée pour étudier la décidabilité. Un problème est décidable si une réponse au problème peut être donnée en temps fini. Elle sert à démontrer par l'absurde que le problème de l'arrêt d'un programme est indécidable.

- f. Supposons maintenant que le tableau d'entrée est trié par ordre croissant. Quelle est la complexité de recherche d'un élément ?
- g. Notez que la fonction Inclue fonctionne sur un tableau trié, mais que dire de la complexité de cet algorithme par rapport à la complexité du problème ?

Visiter le site <https://www.bigocheatsheet.com/> qui met en évidence des problèmes et complexités d'opérations classiques.

2 Classes de complexité des problèmes

Comment savoir si un problème est intrinsèquement complexe, par exemple s'il ne permet pas de solution polynomiale sur une architecture Von Neumann ?

Classification par classes de complexité P (polynomial), NP (non-déterministe polynomial), co-NP (dont le complémentaire est NP) ou NP^{coNP} , $CoNP^{NP}$, $NP^{CoNP \wedge NP}$ etc.

Considérons les problèmes de décision, c'est-à-dire les algorithmes qui répondent OUI/NON, comme c'était le cas pour l'algorithme Inclue. Notons qu'un problème de recherche se transforme en problème de décision, ainsi trouver le plus court chemin dans un graphe, se transforme en existe-t-il un chemin de longueur $\leq k$? Nous voulons classer ces problèmes suivant leur complexité.

Problème de référence : SATISFIABILITY (SAT), et même 3SAT sa version limitée à la satisfiabilité de formule booléenne constituées de conjonctions de disjonctions de 3 littéraux.

Problème **3SAT** : soit une formule booléenne constituée d'une conjonction de disjonctions de 3 littéraux (exemple : (a ou non b ou c) et (non a ou non b ou c)). Existe-t-il une valuation des variables booléennes telle que la formule est vraie ?

Réponse : il nous faut pour le problème général tester toutes les valuations possibles.

- Utiliser un arbre représentant toutes les valuations possibles pour (a ou non b ou c) et (non a ou non b ou c). Au premier niveau on représente a, qui vaut vrai ou faux, b au second, et c au troisième niveau, toutes les valeurs possibles étant représentées au niveau de chaque feuille.
- Quelle est la taille d'un arbre d'une formule constituée de n variables ?
- Si la réponse est OUI, quelle est la complexité de la vérification de cette réponse ?
- Si la réponse est NON, quelle est la complexité de la vérification de cette réponse ?

3SAT est l'un des problèmes **NP-complets**. Il en existe des centaines, recensés notamment dans [1]. Un problème est NP si exhiber un exemple permet de démontrer que la réponse est OUI, alors que démontrer que la réponse est NON requière l'arbre entier.

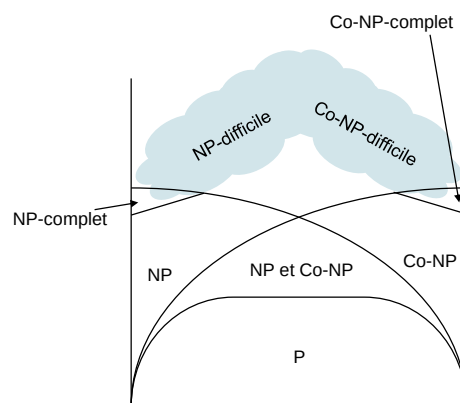


Figure 2 : une représentation possible des classes de complexité P, NP et Co-NP

Les problèmes **Co-NP** sont complémentaires dans le sens où dire NON requière d'exhiber un contre-exemple, alors que dire OUI requière l'arbre entier.

- Le problème CONTRADICTION qui renvoie OUI ssi la formule donnée est une contradiction est-il NP ou Co-NP ?
- Qu'en est-il du problème TAUTOLOGIE ?

La conjecture usuellement assumée est que $P \neq NP$, c'est-à-dire qu'il n'existe pas d'algorithme polynomial (sur architecture Von Neumann) permettant de résoudre des problèmes NP. La preuve de cette conjecture fait partie des 7 problèmes du millénaire.

3 Réduction polynomiale

Etant donné un nouveau problème, pour lequel nous décidons de proposer un algorithme optimal en temps exponentiel, ou une heuristique polynomiale, comment être sûr que l'on ne pourrait pas proposer un algorithme optimal polynomial ?

En démontrant qu'il est aussi dur qu'un problème NP-complet, nous montrons qu'il est **NP-difficile**. Si on souhaite montrer qu'il NP-complet, il faut juste montrer qu'il est NP (vérification réponse donnée en temps polynomial).

Soit le problème d'ordonnancement classique $PD2 | res1.. | C_{max}$ qui signifie minimiser le *makespan* de l'exécution de jobs sur deux processeurs dédiés, sachant que les jobs partagent une ressource. Ce problème arrive lorsqu'on rince les circuits hydrauliques d'un avion civile. Noter que ce problème n'est pas un problème de décision, sa version décisionnelle serait $PD2 | res1.. | C_{max} \leq k$ pour un k donné.

- a. Proposer une stratégie polynomiale résolvant de façon optimale $PD2 | res1.. | C_{max}$

Le souci est que lorsqu'on rince des circuits hydrauliques, il y a des contraintes de précédences entre jobs de façon à ne pas re-salir un tuyau en rinçant un autre. Le problème devient alors $PD2 | res1.., localprecedence | C_{max} \leq k$ qui n'avait jamais été étudié dans la littérature. Notre stratégie ne fonctionne pas, si les précédences ne sont pas consistantes avec elle... Existe-t-il une stratégie optimale polynomiale ?

Considérons le problème NP-complet 3-PARTITION : soit un ensemble de $3n$ entiers positifs, dont la somme totale est nB . Est-il possible de les regrouper en n triplets dont la somme individuelle est B ?

Essayons de montrer que $PD2 | res1.., localprecedence | C_{max} \leq k$ est aussi dur que 3PART en proposant un moyen (qui doit s'exécuter en temps polynomial) de transformer toute instance de problème 3PART en une instance de notre problème, telle que si on pouvait résoudre notre problème en temps polynomial, alors on serait capable de résoudre 3PART en temps polynomial. Cela s'appelle une **réduction polynomiale**.

Soit un problème 3SAT donné sous forme $\{i_1, i_2, \text{etc}, i_{3n}\}$ avec la somme qui vaut nB . Créons une instance de $PD2 | res1.. | C_{max} \leq k$ de la façon suivante :

- Nous supposons deux machines, partageant une ressource r .
- Sur machine 1, nous affectons $3n$ jobs, de durée respective $i_1, i_2, \text{etc}, i_{3n}$ qui utilisent tous la ressource R .
- Sur machine 2, nous affectons B jobs de durée n n'utilisant pas la ressource, et B jobs de durée 1 utilisant la ressource. On introduit des contraintes de précedence entre ces jobs pour forcer les jobs de durée 1 à s'intercaler entre les jobs de durée n .

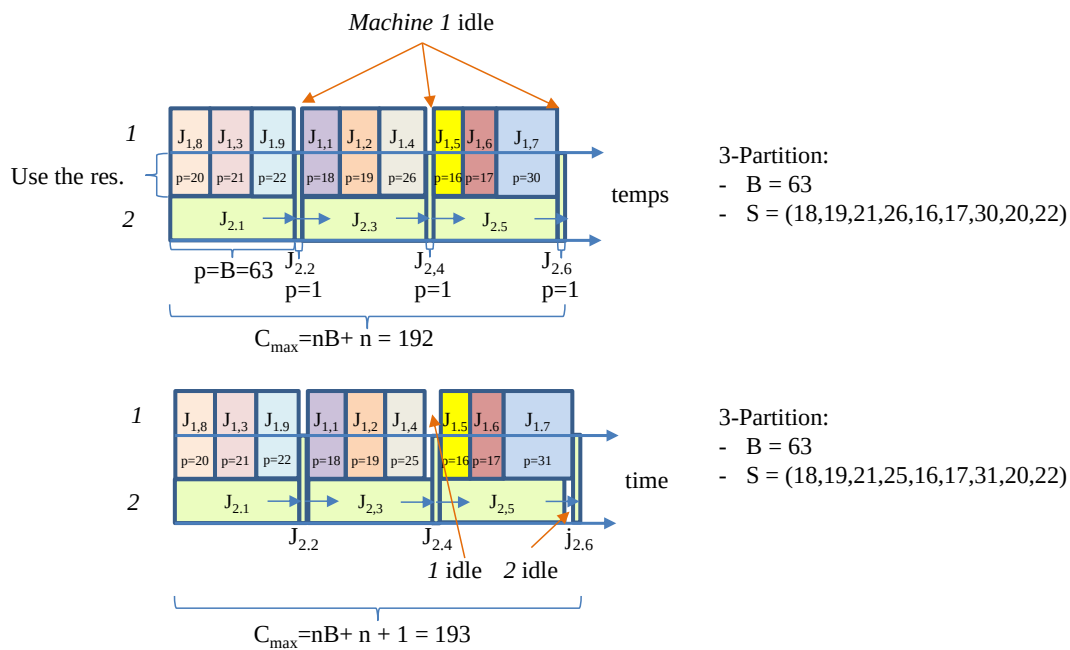


Figure 3 : Modélisation de deux problèmes 3PART sous forme de problèmes PD2 | res1.. | Cmax

b. Observez la Figure 3 et en déduire :

- Quel est le makespan minimal atteignable ?
- Dans quel cas peut-on l'atteindre ?
- Que peut-on en déduire sur la complexité de PD2 | res1.. | Cmax ≤ k ?
- Ce problème est-il NP ?
- Que peut-on en déduire sur sa complexité ?
- Que peut-on en déduire de la complexité de PD2 | res1.. | Cmax ?

4 Sens fort sens faible

Dans la littérature du temps réel, on distingue deux classes de problèmes NP : sens fort et sens faible. Les problèmes NP-difficiles au sens faible sont ceux pour lesquels si l'entrée était donnée en base 1, ils seraient P. On parle de problèmes que l'on peut résoudre en temps pseudo-polynomial.

Par exemple, le crible d'Eratosthène qui permet de décomposer un nombre en facteurs premiers est pseudo-polynomial. Le calcul de temps de réponse de tâches indépendantes ordonnancées par un algorithme à priorités fixes est NP-difficile au sens faible. Vérifier le respect d'échéances d'un système de tâches sous contraintes de ressources partagées est NP-difficile au sens fort.

5 Bibliographie

- [1] M. R. Garey et D. S. Johnson, *Computers and intractability: a guide to the theory of NP-completeness*. in A series of books in the mathematical sciences. New York: W. H. Freeman, 1979.